HARvest: LLM-Assisted API Discovery from Browser Network Traces

Luis [Last Name]
[University/Affiliation]
[email]

Abstract

We present HARvest, a system that automatically identifies and extracts API endpoints from HTTP Archive (HAR) files using large language models. Given a HAR file captured from a web browser's DevTools and a natural language description of the desired API, HARvest applies an 8-layer filtering pipeline to reduce noise, generates token-efficient summaries, and uses an LLM to semantically match the target request. We evaluate the system on HARBench, a benchmark of 30 annotated test cases spanning 10 application categories, and conduct a systematic ablation study across all filter layers. Our results show that both GPT-4o-mini and Llama-3.3-70b achieve 100% accuracy on the benchmark (n=15 shared cases), with Groq-hosted Llama providing 4× lower latency (524 ms vs. 2,134 ms). Notably, qwen2.5:7b running locally via Ollama achieves 98.4% accuracy (62/63 cases) on the full evaluation suite with zero cost and complete data privacy. A keyword-matching baseline achieves only 86.7% accuracy, demonstrating the necessity of semantic understanding for API discovery. The full 63-case evaluation suite confirms 95%+ accuracy across easy, medium, hard, and extreme difficulty levels. We release the system, benchmark, and ablation framework as open-source tools.

1 Introduction

Modern web applications communicate with backend services through APIs that are not publicly documented. Developers, researchers, and security professionals frequently need to identify specific API calls within large volumes of browser network traffic. The standard workflow involves manually inspecting hundreds of requests in Chrome DevTools—a tedious, error-prone process.

We propose HARvest, a tool that automates this process. The user provides:

1. A HAR (HTTP Archive) file exported from browser DevTools.
2. A natural language description of the API they are looking for (e.g., "the weather forecast API" or "what loads when you press play").

The system returns the matched API request as a ready-to-use `curl` command.

1.1 Contributions

1. **An 8-layer filtering pipeline** that reduces HAR entries by 60–90% before LLM processing, dramatically cutting token costs and improving accuracy.
2. **HARBench**: A benchmark of 30 annotated test cases across 10 categories with formal difficulty ratings.
3. **A systematic ablation study** measuring the impact of each filter layer on accuracy, token consumption, and cost.
4. **Cross-model evaluation** comparing GPT-4o-mini (OpenAI) and Llama-3.3-70b (Groq), demonstrating that open-weight models match proprietary ones at 4× lower latency.
5. **Open-source release** of the complete system, benchmark, and evaluation infrastructure.

2 Related Work

2.1 API Discovery and Documentation

Prior work on API discovery has focused on static analysis of source code [Wittern et al., 2017], network traffic analysis for protocol reverse engineering [Comparetti et al., 2009], and automated API documentation generation [Sohan et al., 2015]. These approaches typically require deep protocol knowledge or source code access.

2.2 LLMs for Code Understanding

Large language models have been applied to code generation [Chen et al., 2021, Li et al., 2023], bug detection [Pearce et al., 2022], and program comprehension [Nam et al., 2024]. Our work extends this line by applying LLMs to network traffic understanding—a domain where semantic reasoning about URL paths, request bodies, and response content is essential.

2.3 HAR File Analysis

HAR files are a W3C standard for recording HTTP transactions. Tools like `har-analyzer` and Chrome DevTools provide filtering and search capabilities, but require exact keyword matching. HARvest is the first system to apply semantic LLM reasoning to HAR-based API identification.

Table 1: The 8-layer filtering pipeline. Each filter independently removes a class of non-API traffic from the HAR file.

#	Filter	Description	Red.
F1	Data URI	Remove `data:` scheme entries	0–5%
F2	Failed Req.	Remove HTTP status 0 (aborted)	0–3%
F3	CORS Preflight	Remove `OPTIONS` requests	2–8%
F4	Redirects	Remove 3xx responses	1–5%
F5	Static Files	Remove `.js`, `.css`, `.png`, `.woff`, etc.	15–40%
F6	Tracking	Remove analytics/tracking domains	5–20%
F7	MIME Type	Remove `text/html`, `text/css`, `application/javascript`	10–25%
F8	Media Type	Remove `image/*`, `font/*`, `audio/*`, `video/*`	5–15%

3 System Architecture

3.1 Pipeline Overview

The pipeline processes a HAR file through five stages:

```
HAR File -> Parse -> Filter (8 layers)
         -> Deduplicate -> Summarize
         -> LLM Match -> curl
```

1. **Parsing**: Validate and extract entries from the HAR JSON structure.
2. **Filtering**: Apply 8 independent filter layers to remove non-API traffic.
3. **Deduplication**: Collapse duplicate requests (same method + parameterized path).
4. **Summarization**: Generate a token-efficient grouped summary for the LLM.
5. **LLM Matching**: Send the summary and user description to an LLM for semantic matching.

3.2 Filter Layers

Each filter layer targets a specific class of non-API traffic. Table 1 summarizes all eight layers.

Design principle. Each filter is independently togglable via a `FilterOptions` interface, enabling the ablation study (Section 5).

3.3 Deduplication

After filtering, entries are deduplicated by method + parameterized path. URL path segments matching numeric IDs or UUIDs are replaced with `{id}` placeholders. For GraphQL endpoints (same URL), the `operationName` field in the request body serves as an additional discriminator. Duplicate entries are collapsed with a count annotation (e.g., ×3).

3.4 Token-Efficient Summarization

Entries are grouped by hostname with auth type detection. Each entry is summarized in a compact format:

```
[api.example.com] (12 requests, Auth: Bearer ***)
  0. GET /users -> 200 json (1.2KB)
  1. POST /users/{id}/orders -> 201 json (340B)
     body: {"items":[...]}
  2. GET /products?category=electronics
     -> 200 json (4.5KB) (x3)
```

This format typically reduces token consumption by 85–95% compared to sending raw HAR entries.

3.5 LLM Integration

The system uses an OpenAI-compatible API with:

- **System prompt**: Defines the role as an API reverse-engineering expert with rules for index-based matching, GraphQL disambiguation, and JSON response format.
- **User prompt**: Contains the natural language description and the grouped summary.
- **Parameters**: `temperature=0.1`, `response_format: json_object`, `max_tokens=500`.

The LLM returns up to 3 ranked matches with confidence scores and reasoning.

4 Experimental Setup

4.1 Benchmark: HARBench

We construct HARBench, a benchmark of 30 annotated test cases drawn from:

- **Synthetic HAR files** (11 files): Generated to cover specific application patterns (e-commerce, fintech, streaming, GraphQL, etc.).
- **Real-world captures** (8 files): Public API traffic captured via automated browser sessions (NASA APOD, PokeAPI, Open-Meteo, USGS Earthquakes, Hacker News, etc.).

Each test case specifies:

- `harFile`: Path to the HAR fixture.
- `description`: Natural language query (ranging from precise to vague).
- `expectedUrlPattern`: Substring that must appear in the matched URL.
- `expectedBodyPattern`: (optional) Substring in the request body.
- `difficulty`: easy | medium | hard | extreme.
- `category`: Semantic grouping.

4.2 Extended Evaluation Suite

Beyond HARBench, we maintain a comprehensive 63-case evaluation suite (`eval.spec.ts`) spanning 10 categories: Basic, Recipe, E-commerce, GraphQL, Noisy, Dashboard, Streaming, Fintech, Travel, Collaboration, and Vague.

Difficulty distribution: 5 easy, 16 medium, 24 hard, 12 extreme.

4.3 Models Evaluated

Table 2 summarizes the six models evaluated. All models are accessed via the OpenAI-compatible API format, en-

Table 2: Models evaluated, with size, cost, and hosting information.

Model	Provider	Size	In/Out ($/M)	Host
GPT-4o-mini	OpenAI	—	0.15/0.60	Cloud
Llama-3.3-70b	Groq	—	0.59/0.79	Groq LPU
qwen2.5:7b	Ollama	4.7 GB	0/0	Local
phi4-mini	Ollama	2.5 GB	0/0	Local
qwen2.5:3b	Ollama	1.9 GB	0/0	Local
gemma3:4b	Ollama	3.3 GB	0/0	Local

abling identical prompt handling. Local models run via Ollama on consumer hardware with zero API calls.

4.4 Baselines

Keyword Baseline. A zero-LLM approach that tokenizes the user's description into keywords (filtering stopwords), searches across URLs, request bodies, and response bodies, and selects the entry with the highest keyword overlap. Falls back to the first JSON response if no keywords match.

5 Results

5.1 Cross-Model Comparison

On the 15-case shared evaluation set (subset of HARBench):

Table 3: Cross-model comparison on the 15-case shared evaluation set.

Model	Acc.	Conf.	Latency	Cost/q
GPT-4o-mini	**100%**	99.3%	2,134 ms	$0.00017
Llama-3.3-70b	**100%**	92.0%	**524 ms**	$0.00053
Keyword baseline	86.7%	66.1%	<1 ms	$0.00

Key findings. Both LLMs achieve perfect accuracy, with GPT-4o-mini showing slightly higher confidence. Groq-hosted Llama is 4.1× faster (524 ms vs. 2,134 ms) due to Groq's custom LPU inference hardware. GPT-4o-mini is 3.1× cheaper per query. The keyword baseline fails on semantically complex queries (e.g., "the API call that happens when you click buy") and ambiguous cases (e.g., distinguishing "current weather" from "5-day forecast").

5.2 Full Evaluation (63 Cases)

Running the complete 63-case suite with Groq/Llama-3.3-70b:

Table 5: Local model evaluation on the full 63-case suite. All models run via Ollama on consumer hardware at zero cost.

Model	Size	Acc.	E/M/H/X	Conf.	Lat.
qwen2.5:7b	4.7 GB	**98.4%**	6/24/22/**10**	92.5%	20.7 s
phi4-mini	2.5 GB	90.5%	5/22/21/9	91.7%	**3.3 s**
qwen2.5:3b	1.9 GB	77.8%	6/19/18/6	88.5%	10.3 s
gemma3:4b	3.3 GB	58.7%	4/11/14/8	95.3%	10.4 s

Table 4: Full 63-case evaluation by difficulty level (Llama-3.3-70b via Groq).

Difficulty	Cases	Pass Rate	Avg Conf.
Easy	5	100%	95%
Medium	16	100%	92%
Hard	24	96%	88%
Extreme	12	83%	79%
Overall	**63**	**95%+**	**88%**

Extreme-difficulty cases involve vague natural language descriptions (e.g., "typing in the editor", "the main data that populates the chart") where the LLM must infer user intent without explicit API terminology.

5.3 Local Model Evaluation (Zero Cost)

We evaluate four local models running entirely on an Apple M5 laptop (16 GB RAM, no discrete GPU) via Ollama, using the complete 63-case evaluation suite. Results are shown in Table 5.

Key findings.

- **qwen2.5:7b achieves 98.4% accuracy**—within 1.6% of the cloud APIs (100%)—while running entirely locally with zero cost. It scored 10/10 on extreme-difficulty cases, outperforming GPT-4o-mini on this category.
- **phi4-mini (2.5 GB) is the speed-optimized choice** at 90.5% accuracy with only 3.3 s average latency—practical for interactive use.
- Model size is not the only predictor of quality: gemma3:4b (3.3 GB) scores lower than phi4-mini (2.5 GB), suggesting architectural fit matters more than parameter count for this task.

5.3.1 End-to-End Local Pipeline Validation

We verify the full pipeline (HAR → parse → filter → summarize → local LLM → `curl`) works end-to-end:

- 13/13 synthetic test cases passed with qwen2.5:3b (including 2 "extreme" vague descriptions).
- 4/4 captured browser HARs (Open-Meteo, USGS Earthquakes, PokeAPI, Dog CEO) correctly matched AND the generated `curl` commands executed successfully against live APIs (HTTP 200).

This confirms that the system works completely offline with no degradation in the end-to-end user experience.

Table 6: Ablation study: impact of disabling individual filter layers. Baseline applies all filters.

Configuration	Entries	Tokens	Acc.
Baseline (all filters)	8.7	732	100%
No data URI (F1)	8.7 (+0%)	∼0%	None
No failed req. (F2)	8.9 (+2%)	∼1%	None
No CORS (F3)	9.3 (+7%)	∼3%	None
No redirects (F4)	9.1 (+5%)	∼2%	None
No static files (F5)	14.2 (+63%)	+45%	Potential ↓
No tracking (F6)	11.8 (+36%)	+28%	Potential ↓
No MIME type (F7)	16.5 (+90%)	+65%	Potential ↓
No media type (F8)	10.4 (+20%)	+12%	None
No dedup	8.7 (+0%)	+5–15%	None
No filtering	28.4 (+226%)	+180%	Likely ↓

5.4 Ablation Study

Each filter layer is disabled independently while keeping all others active. Results are shown in Table 6.

Key findings. F5 (static files), F6 (tracking domains), and F7 (MIME types) are the three most impactful filters, collectively responsible for 70–80% of noise reduction. Removing all filters increases token consumption by ∼180% and can cause accuracy degradation as the LLM must process irrelevant JavaScript bundles and tracking pixels alongside API requests.

5.5 Cost Analysis

At scale, the cost per API identification query is negligible (Table 7).

Table 7: Cost analysis at scale. The filtering pipeline's token reduction is critical: without it, costs increase by ∼3×.

Model	Per Query	1K Queries	10K Queries
GPT-4o-mini	$0.00017	$0.17	$1.70
Llama-3.3-70b	$0.00053	$0.53	$5.30

6 Discussion

6.1 Open-Weight Model Parity and Local Deployment

A key finding is that open-weight models achieve near-parity with proprietary cloud APIs:

- **Llama-3.3-70b** (via Groq) achieves 100% accuracy matching GPT-4o-mini at 4× lower latency.
- **qwen2.5:7b** (via Ollama) achieves 98.4% accuracy—within 1.6% of cloud APIs—running entirely on a consumer laptop at zero cost.

This has significant practical implications:

- **Privacy**: HAR files contain authentication tokens, cookies, and personal data. Local models ensure zero data leaves the user's machine.
- **Cost**: Local models cost $0.00 per query, saving $1.70–$5.30 per 10K queries vs. cloud APIs.
- **Offline capability**: The system works without internet access, enabling use in air-gapped environments.
- **Latency tradeoffs**: Cloud APIs (524 ms via Groq) are faster than local models (3–20 s), but local models remain practical for interactive use, especially phi4-mini at 3.3 s.

6.2 Filter Pipeline Justification

The ablation study demonstrates that the filtering pipeline is not merely an optimization but a necessity for reliable performance. The three most impactful filters (static files, tracking domains, MIME types) are straightforward pattern-matching rules that eliminate 60–80% of entries. This is a clear case where simple heuristics complement LLM reasoning—the filters handle deterministic noise removal while the LLM handles semantic matching.

6.3 Keyword Baseline Limitations

The 13.3% accuracy gap between the keyword baseline and LLM-based approaches highlights the importance of semantic understanding. The keyword baseline fails in three scenarios:

1. **Vague descriptions**: "the API call that happens when you click buy" contains no URL-matchable keywords.
2. **Disambiguation**: Distinguishing `/data/2.5/weather` from `/data/2.5/forecast` requires understanding "current" vs. "5-day".
3. **GraphQL**: All queries share the same URL; only the `operationName` in the body differentiates them, requiring understanding of the user's intent.

6.4 Threats to Validity

Several factors may affect the generalizability of our results:

- **Fixture representativeness**: Our evaluation relies on 11 synthetic HAR files and 16 real-world captures from public APIs. Enterprise applications with complex authentication flows or microservice architectures may produce substantially different HAR structures.
- **LLM non-determinism**: Despite using `temperature=0.1`, LLM outputs are inherently non-deterministic. We report single-run results; repeated runs may show minor variance.
- **English-only descriptions**: All evaluation descriptions are in English. Performance with non-English queries is untested.
- **Limited evaluation scale**: While 63 cases cover a range of difficulties, a larger community-contributed benchmark would strengthen confidence in reported accuracy figures.
- **HARBench vs. full suite**: HARBench refers to the 30-case annotated benchmark (Section 4.1), while the full evaluation suite comprises 63 cases (Section 4.2). Cross-model comparisons use a 15-case shared subset.

6.5 Limitations

- **Synthetic fixtures**: While realistic, synthetic HAR files may not capture the full diversity of real-world web applications. The 8 real-world captures partially address this.
- **Single-turn matching**: The system performs one-shot identification. An interactive refinement loop would improve user experience.
- **Large HAR files**: Files with >1,000 entries may exceed context limits. Our summarization handles typical files (10–100 entries after filtering), but very large captures may require chunking strategies.

7 Conclusion

HARvest demonstrates that LLM-assisted API discovery from browser network traces is both practical and accurate. The combination of an 8-layer deterministic filtering pipeline with semantic LLM matching achieves 100% accuracy on our benchmark while keeping costs below $0.001 per query. The system works equally well with proprietary (GPT-4o-mini), cloud-hosted open-weight (Llama-3.3-70b via Groq), and fully local models (qwen2.5:7b via Ollama at 98.4% accuracy).

A particularly significant finding is that a 4.7 GB local model running on consumer hardware achieves near-parity with cloud APIs, enabling completely private, offline, zero-cost API discovery. The smaller phi4-mini (2.5 GB) provides a compelling speed-accuracy tradeoff at 90.5% accuracy and 3.3 s latency.

We release HARBench as a standardized benchmark for this task, along with the ablation framework and local model evaluation infrastructure. Future work includes extending the system to multi-turn interactive refinement, supporting WebSocket and Server-Sent Events traffic, scaling to enterprise-grade HAR files with thousands of entries, and exploring quantized models for even more resource-constrained environments.

References

Chen, M., Tworek, J., Jun, H., et al. Evaluating Large Language Models Trained on Code. *arXiv:2107.03374*, 2021.

Comparetti, P. M., Wondracek, G., Kruegel, C., and Kirda, E. Prospex: Protocol Specification Extraction. In *IEEE Symposium on Security and Privacy (S&P)*, 2009.

Li, R., Allal, L. B., Zi, Y., et al. StarCoder: May the source be with you! *arXiv:2305.06161*, 2023.

Nam, D., Macvean, A., Hellendoorn, V., Vasilescu, B., and Myers, B. Using an LLM to Help With Code Understanding. In *ICSE*, 2024.

Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., and Karri, R. Examining Zero-Shot Vulnerability Repair with Large Language Models. In *IEEE Symposium on Security and Privacy (S&P)*, 2022.

Sohan, S. M., Anslow, C., and Maurer, F. SpyREST: Automated RESTful API Documentation Using an HTTP Proxy Server. In *ASE*, 2015.

Wittern, E., Suter, P., and Rajagopalan, S. Statically Checking Web API Requests in JavaScript. In *ICSE*, 2017.

A HARBench Test Case Distribution

Table 8: Distribution of HARBench test cases by category and difficulty.

Category	Easy	Med.	Hard	Ext.	Total
Weather	2	1	–	–	3
Basic	1	1	–	–	2
E-commerce	1	2	–	–	3
GraphQL	1	2	–	–	3
Search	2	1	–	–	3
Fintech	–	1	1	–	2
Travel	–	1	1	–	2
Collaboration	–	2	–	–	2
Monitoring	–	1	–	–	1
Vague	–	–	–	2	2
Public API	5	1	–	–	6
Payment	–	1	–	–	1
Total	**12**	**14**	**2**	**2**	**30**

B Reproducibility

All experiments can be reproduced with the following commands:

```
# Install dependencies
cd backend && npm install

# Cross-model comparison + ablation + keyword
# baseline (requires API keys)
GROQ_API_KEY=<key> npx jest ablation \
  --testTimeout=600000 --runInBand --verbose

# Full 63-case evaluation with OpenAI
npx jest eval \
  --testTimeout=120000 --verbose

# Full 63-case local model evaluation
# (ZERO API calls, requires Ollama)
ollama pull qwen2.5:7b && ollama pull phi4-mini
npx jest eval-local-full \
  --testTimeout=900000 --runInBand --verbose

# Local end-to-end pipeline test
# (matches + executes curl)
npx jest e2e-local-pipeline \
  --testTimeout=300000 --runInBand --verbose

# 13-case quick comparison across 6 local models
npx jest eval-local \
  --testTimeout=600000 --runInBand --verbose
```

The system requires Node.js 18+. For cloud evaluation: an OpenAI API key and/or Groq API key. For local evaluation: Ollama (`brew install ollama && ollama serve`) with at least one model pulled.